

Architecture Process

Introduction

The architecture described herein pertains to a system designed to handle optimization tasks. The system is composed of multiple components, each with specific responsibilities and dependencies. The boundaries of the system are defined by the interactions between these components, which are primarily concerned with processing optimization requests. The scope of this document is to provide a detailed overview of the system's architecture, focusing on its elements, interfaces, and data flow.

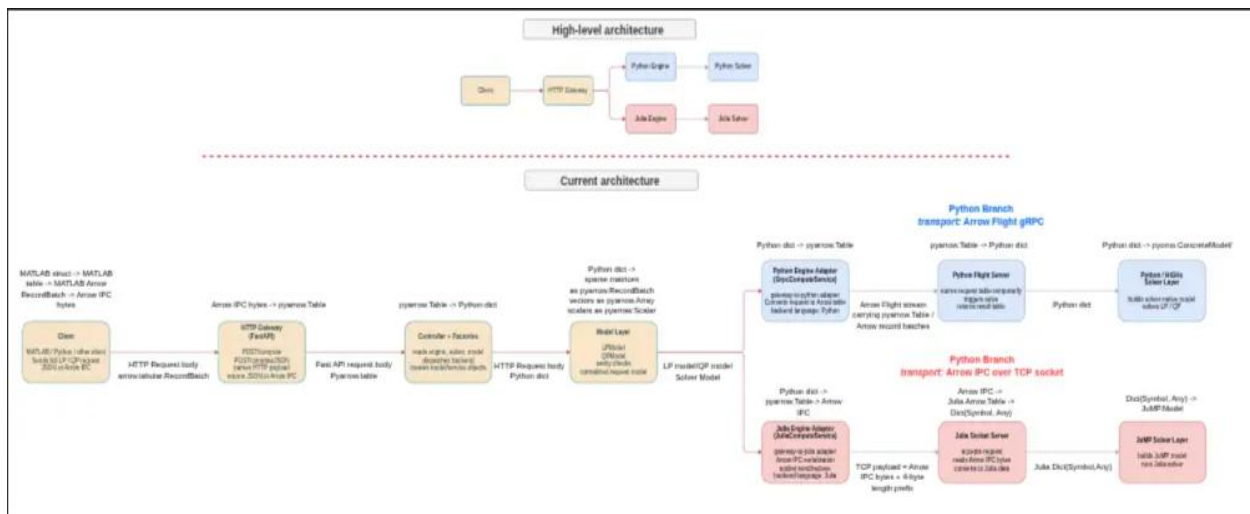
System Elements and Components

The system is initiated by the Client, which is responsible for sending optimization task requests. These requests are directed to the HTTP Gateway, implemented using FastAPI, which serves as the entry point for incoming HTTP requests. The HTTP Gateway routes these requests to the appropriate processing engines, namely the Python Engine and the Julia Engine. Each engine is responsible for processing optimization tasks using their respective solvers: the Python Solver and the Julia Solver.

The Controller + Factories component plays a crucial role in reading engine, solver, and model information to dispatch backend tasks and create model/service objects. It relies on the Model Layer, which performs sanity checks and normalizes request models. The Model Layer supports two types of models: LPModel and QPModel.

The Python Engine Adaptor and Julia Engine Adaptor are responsible for converting requests into formats suitable for backend processing. The Python Engine Adaptor uses the GrpcComputeService to convert requests to Arrow table format, while the Julia Engine Adaptor uses the JuliaComputeService to serialize requests into Arrow IPC format.

The Python Flight Server and Julia Socket Server handle the storage and processing of requests. The Python Flight Server temporarily stores request tables, triggers the solve process, and returns result tables. The Julia Socket Server accepts requests, reads Arrow IPC bytes, and converts them into Julia data for further processing by the JuMP Solver Layer.



Interfaces and Protocols

The primary interface between the Client and the HTTP Gateway is an HTTP POST mechanism, where the payload can be either JSON or Arrow IPC. This interaction is unidirectional, from the Client to the Server. Within the server, the HTTP Gateway communicates with the Controller + Factories through function calls, passing parsed HTTP payloads.

The Controller + Factories further interacts with the Model Layer via function calls, transmitting model/service objects. The Model Layer then communicates with the Python Engine Adaptor and Julia Engine Adaptor through function calls, delivering normalized request models.

The Python Engine Adaptor and Julia Engine Adaptor interface with their respective servers using Arrow Table and Arrow IPC mechanisms, respectively. These interactions are also unidirectional, from the adaptors to the servers.

Data Flow

The data flow begins with the Client sending a request in JSON or Arrow IPC format to the HTTP Gateway. The HTTP Gateway parses this payload and routes it to the Controller + Factories, which dispatches the request to the Model Layer. The Model Layer performs sanity checks and normalizes the request model, converting it into an Arrow table for the Python Engine Adaptor or serializing it into Arrow IPC for the Julia Engine Adaptor.

The Python Engine Adaptor converts the normalized request model into an Arrow table, which is then temporarily stored by the Python Flight Server. The server triggers the solve process and returns the result table. Similarly, the Julia Engine Adaptor serializes the request into Arrow IPC, which is read by the Julia Socket Server and converted into Julia data for processing by the JuMP Solver Layer.